Robot de Réconfort - avancement du binôme

Mis à jour le 29/10/2026 - projet global : 54 %

Prochaines étapes

Axel : memoire.cpp, puis agent.cpp (decider, appliquer, bilan).
Eliott : main.cpp avec Axel (boucle + trace), puis validation 5.6 et affichage texte.

Axel (B)

Agent & décision - 79 % : 10 fait, 2 en cours, 2 à faire, 0 bloqué

Carte & mémoire

FAIT — Carte mentale (hypothèse optimiste)
carte_robot.cpp/.hpp : INCONNUE/LIBRE/OCCUPEE, mise à jour par case

EN COURS — Mémoire du robot (struct Memoire)
memoire.hpp écrit (carte, robot, sélecteur, vu/contenu) ; memoire.cpp à écrire

EN COURS — Mise à jour à chaque perception
noter_perception / oublier_armoire déclarés, non implémentés

Planification

FAIT — BFS + file (plus court chemin)
graphe.cpp : prochain_pas, file écrite à la main, sans lib

FAIT — Replanification à chaque pas
un pas puis recalcul ; PAS_DE_CHEMIN si inaccessible

Décision

FAIT — Lecture du dictionnaire (6.1)
dictionnaire.cpp : premier mot reconnu, sinon indéterminée

FAIT — Choix du casier (6.2) : 15 candidats triés
choix_casier.cpp : casiers_a_essayer, prochain_casier, étiquettes de repli

FAIT — Itinéraire du sélecteur (annexe C)
pas_du_selecteur : vertical d'abord + bouclage des colonnes (C.3, accepté par C.4)

A FAIRE — Enchaînement des 4 étapes par demande
agent.cpp : classe Agent de interface.hpp (decider, appliquer, bilan)

A FAIRE — Gestion des échecs (6.3)
résident inaccessible, armoire vide, émotion indéterminée

Tests

FAIT — Test 1 : planification petite grille
tests.cpp : mini-grille B.3, chemin SSEENE, puis X si bouché

FAIT — Test 2 : replanification (mur découvert)
tests.cpp : murs vus en (1,2), nouveau chemin OSSEENE

FAIT — Test 3 : lecture du dictionnaire
tests.cpp : 8 émotions + 2 mots connus + aucun mot

FAIT — Test 5 : règle de repli
tests.cpp : exact, intensité, voisine, armoire vide, ordre respecté

Eliott (A)

Monde & validation - 50 % : 11 fait, 0 en cours, 11 à faire, 0 bloqué

Roue

FAIT — roue_avancer : bouclage dans [0, n-1]
roue.cpp : pas négatifs gérés

FAIT — roue_distance : plus court côté
roue.cpp : résultat dans [0, n/2]

Environnement

- FAIT — struct Environnement::Etat
  grille, robot, sélecteur, armoire 2D (optional), positions fixes, résidents, objet porté, résident visé, départs
- FAIT — Makefile : environnement.o dans ROBOT
  conflit de fusion réparé
- FAIT — Destructeur ~Environnement()
  = default, après la struct
- FAIT — Constructeur (lecture carte + armoire)
  grille, 6 positions, résidents, armoire 3x8 (casiers null gérés)
- FAIT — connaissances()
  tout SAUF la grille et les objets
- FAIT — percevoir()
  4 voisins N/S/E/O, devant_armoire, contenu_casier
- FAIT — viser_resident(), robot(), selecteur()
- FAIT — executer() : légalité des actions
  les 6 actions ; FIN refusée (default)

Validation 5.6

- A FAIRE — Fichier absent / illisible / pas UTF-8, JSON invalide
- A FAIRE — Champ obligatoire absent ou mauvais type
  dans les 4 fichiers
- A FAIRE — Grille non rectangulaire, dimensions, position hors grille
- A FAIRE — Symboles P / A / D / R cohérents avec les positions
- A FAIRE — Résidents en double, autre carte, résident absent
- A FAIRE — Casiers : hors armoire, doublon, étiquette, casier_depart
- A FAIRE — Dictionnaire : émotion/intensité inconnue, mot en double
- A FAIRE — Erreur -> stderr, code non nul, aucune trace

Affichage

- A FAIRE — Grille redessinée à chaque pas
  robot + murs découverts (débogage, vidéo)

Tests

- FAIT — Test 4 : sélecteur, bouclage colonne 7 <-> 0
  tests.cpp : test_selecteur, 8 vérifications
- A FAIRE — Test 6 : un fichier cassé par catégorie de 5.6
- A FAIRE — Test 7 : résident muré (avec Axel)

Ensemble

À la fin, à deux - 0 % : 0 fait, 0 en cours, 5 à faire, 0 bloqué

Intégration

- A FAIRE — main.cpp : boucle Environnement <-> Agent
  trace, livraisons, résumé

Relecture

- A FAIRE — Relecture croisée
  Eliott : agent/graphe/carte_robot ; Axel : environnement/roue

Rendu

- A FAIRE — Rapport 3 pages
  architecture, replanification, repli, 3 échecs
- A FAIRE — Vidéo 1 min 30
  robot qui se cogne et replanifie
- A FAIRE — README + rendu.json
  commande de test ; C++, make, commande, vidéo